DAY 16

Software Architecture and Architectural Design

Software Engineering & Project Management • Student Learning Book

Learning outcomes

By the end of this chapter, you should be able to:

• Explain software architecture and why it matters before detailed coding begins.

• Identify components, connectors, interfaces, and architectural constraints.

• Describe common architectural styles: layered, client–server, Model–View–Controller (MVC), repository, and pipe-and-filter.

• Compare architectural choices using quality attributes such as maintainability, performance, security, scalability, and testability.

• Sketch a high-level architecture for a College Event Management System (CEMS).

Running case study: College Event Management System (CEMS). Students discover events, register, receive confirmations, and organizers manage event details and attendance.

1. What is Software Architecture?

Software architecture is the high-level structure of a software system. It describes the major parts of the system, the responsibilities assigned to those parts, how they communicate, and the important rules that guide their interactions.

Think of a building. An architect decides where rooms, stairs, lifts, water lines, and electrical systems should go before construction details are finalized. Similarly, a software architect decides the major software parts and how they fit together before developers implement every class and function.

Architecture versus detailed design

Aspect	Software architecture	Detailed design
Focus	Major subsystems, boundaries, communication, deployment	Classes, methods, algorithms, data structures
Typical question	Should registration and payment be separate services?	How should the registerForEvent() method validate a request?
Main audience	Architects, technical leads, developers, operations and stakeholders	Developers and testers implementing specific components
Typical artefacts	Context diagram, component diagram, deployment view, architecture decision record	Class design, method contracts, pseudocode, detailed sequence logic

Important: Architecture does not replace detailed design. It provides the structure within which detailed design and coding take place.

2. Building Blocks of an Architecture

2.1 Components

A component is a major part of the system that performs a defined responsibility. Examples include a user interface, authentication module, event service, notification module, and database.

2.2 Connectors

A connector describes how components interact. Examples include an HTTP request, a function call, a message queue, or a database connection.

2.3 Interfaces

An interface defines the services or operations a component makes available to other components. A registration component might expose an operation such as `registerStudent(eventId, studentId)`.

2.4 Constraints

Constraints are rules that limit architectural choices. Examples: the college requires single sign-on; student data must be protected; the system must run in an approved cloud environment; or the team must use an existing database.

> A useful architecture description answers four questions: What are the parts? What does each part own? How do the parts communicate? What rules must the design satisfy?

3. Common Architectural Styles

3.1 Layered architecture

The system is organized into layers, with each layer offering services to the layer above it. A common web application arrangement is presentation, application/business logic, data access, and database.

Layer	Responsibility	CEMS example
Presentation	Displays information and accepts user input	Event list, registration form, organizer dashboard
Application / business	Applies use cases and business rules	Check capacity, prevent duplicate registration
Data access	Reads and writes persistent data	Queries events and saves registrations
Database	Stores durable records	Students, events, registrations, attendance

Illustration (top to bottom):

```
Student / Organizer
↓
Presentation layer
↓
Application and business rules
↓
Data access layer
↓
Database
```

Advantages: clear separation of concerns, easier onboarding, and testable boundaries. Limitations: excessive layering can add overhead; poorly designed layers may become tightly coupled.

3.2 Client–server architecture

Clients request services from a server. A browser or mobile application is a client; the backend API is a server. The server may validate a request, apply business rules, and retrieve or update data.

CEMS example: a student's browser requests the event list from an API. The API fetches approved events from the database and returns a response.

Advantages: centralized business rules and data management. Limitations: network failures and server capacity can affect users; security must be enforced on the server, not only in the browser.

3.3 Model–View–Controller (MVC)

MVC separates an application's data and rules (Model), user-facing representation (View), and request/control flow (Controller). The exact implementation varies by framework.

Part	Role	CEMS example
Model	Represents data and domain rules	Event, Student, Registration
View	Presents information to the user	Event page, registration confirmation
Controller	Receives input and coordinates the response	RegistrationController handles a submit action

MVC can help teams change the interface without rewriting all business rules. It does not automatically guarantee clean architecture: controllers can still become overloaded if responsibilities are not separated.

3.4 Repository architecture

In a repository style, several components share a central data store. For example, the event catalogue, organizer dashboard, and reporting module may all access a shared event database through defined interfaces.

Advantages: consistent shared data and straightforward reporting. Risks: the repository can become a bottleneck or a single point of failure, and direct access by many components can create tight coupling.

3.5 Pipe-and-filter architecture

Data passes through a sequence of processing stages called filters. Each filter transforms or checks the data and passes the result through a pipe to the next stage.

Example: an event-import tool reads a CSV file → validates columns → checks duplicate event IDs → normalizes dates → writes approved records to the database.

This style is useful for data-processing workflows. It is less natural for interactive screens where many operations depend on user actions.

4. Choosing an Architectural Style

There is no universally best style. Select an architecture by understanding the system's requirements, constraints, team skills, and expected growth.

Need / constraint	Possible architectural direction	Reason and caution
Simple business application with clear responsibilities	Layered	Easy to understand; avoid unnecessary layers
Web or mobile app talking to a backend	Client–server	Separates user interface from trusted server logic
Framework-based web application	MVC	Separates model, view, and request handling

Need / constraint	Possible architectural direction	Reason and caution
Shared records and reporting across modules	Repository	Centralized data; protect against bottlenecks and uncontrolled coupling
Batch imports or sequential data transformations	Pipe-and-filter	Stages can be tested separately

Architecture decisions are trade-offs. A choice that improves one quality attribute may make another harder. For example, adding more independent services may improve independent scaling but increase deployment, monitoring, and communication complexity.

5. Quality Attributes

Quality attribute	Meaning	CEMS design question
Maintainability	How easily can the system be changed?	Can the event-registration rule be updated without rewriting every screen?
Performance	How quickly does the system respond?	Can students load the event list quickly during registration opening?
Scalability	How well can it handle growth?	Can the system handle a sudden rush of registrations?
Security	How well does it protect users and data?	Can only authorized organizers edit event details?
Availability	How consistently is it usable?	What happens if a server or database connection fails?
Testability	How easily can behavior be verified?	Can registration rules be tested without launching the entire application?
Usability	How easily can users complete tasks?	Can students register on a phone without confusion?

Quality attributes should be written as concrete scenarios where possible. Instead of saying 'the system must be fast', specify a measurable target such as 'the event list should load within two seconds for 95% of requests under the agreed test load.' The exact target must come from stakeholder needs and realistic testing.

6. CEMS High-Level Architecture

The following is one reasonable starting design, not the only valid solution.

```
Student / Organizer browser or mobile interface
↓ HTTPS requests
Backend API / Controller layer
↓
Application services: Authentication • Event management • Registration • Attendance
↓
Data access interfaces
↓
Database: Users • Events • Registrations • Attendance
■ Notification adapter → Email/SMS provider
```

Explanation: the interface collects user input, the backend validates permissions and business rules, data-access components persist records, and the notification adapter sends confirmations through an external provider. In a small

college project, these can be modules in one deployable application; they do not need to be separate microservices.

Example registration flow:

1. A student selects an event and submits the registration form.
2. The backend checks authentication and authorization.
3. The registration service checks event capacity and duplicate registration rules.
4. The data layer saves the registration transactionally.
5. The system returns a confirmation and may request an email notification.

Failure case: if the event is full, the system should return a clear message and must not create an invalid registration. If notification delivery fails after registration succeeds, the architecture should define whether to retry the notification without duplicating the registration.

7. Architecture Decisions and Documentation

Important architecture decisions should be recorded so that the team understands not only what was chosen, but why.

Decision record field	Example
Context	Students and organizers use a web application for event registration.
Decision	Start with a layered monolithic application.
Reasons	Small team, moderate expected traffic, simpler deployment and debugging.
Consequences	Modules must have clear boundaries; later extraction into services may require refactoring.
Alternatives considered	Separate microservices; server-rendered application; client–server API.

A useful architecture document may include a system context diagram, component diagram, deployment view, key quality scenarios, interfaces, security assumptions, constraints, and decision records.

8. Common Mistakes

• Starting with fashionable technologies before understanding the requirements.

• Drawing boxes without defining responsibilities or communication between them.

• Putting all business logic in controllers or user-interface code.

• Assuming the browser is a trusted place to enforce security.

• Creating microservices for a small system without a clear need.

• Ignoring deployment, backup, monitoring, failure handling, and data protection.

• Treating the architecture diagram as permanently fixed even when requirements change.

9. Classroom Activity: Design the CEMS Architecture

Work in groups. Create a one-page architecture for the College Event Management System.

1. List at least five major components and state the responsibility of each.
2. Choose a style or combination of styles and justify the choice.
3. Draw arrows showing the important interactions and label the communication method.

4. Identify three quality attributes and write one measurable scenario for each.

5. Describe two failure cases and how the system should respond.

6. Record one architecture decision, one alternative, and the trade-off.

> **Deliverable:** one architecture diagram, a component responsibility table, and a short decision record.

10. Review Questions

1. Define software architecture.

Suggested answer: It is the high-level structure of a system, including its major components, responsibilities, interactions, and constraints.

2. How does architecture differ from detailed design?

Suggested answer: Architecture focuses on system-level structure and boundaries; detailed design focuses on classes, methods, algorithms, and implementation details.

3. What is a connector? Give two examples.

Suggested answer: A mechanism for interaction between components, such as an HTTP request or a database connection.

4. Explain layered architecture using CEMS.

Suggested answer: Presentation, business/application logic, data access, and database layers each have distinct responsibilities.

5. How do MVC's parts differ?

Suggested answer: Model holds data/domain behavior, View presents information, and Controller coordinates input and responses.

6. Give one benefit and one risk of repository architecture.

Suggested answer: Benefit: shared consistent data; risk: bottleneck or tight coupling around the central store.

7. When is pipe-and-filter useful?

Suggested answer: When data is processed through a sequence of independent transformations, such as validating and importing a CSV file.

8. Why is 'fast' not a complete performance requirement?

Suggested answer: It lacks a measurable threshold, workload, and conditions under which performance is evaluated.

9. Why might microservices be unnecessary for a small college project?

Suggested answer: They add deployment, communication, monitoring, and operational complexity that may not be justified.

10. What should an architecture decision record contain?

Suggested answer: Context, decision, reasons, alternatives, and consequences.

11. Glossary

Term	Meaning
Architecture	High-level structure and key interactions of a software system
Component	A major unit with a defined responsibility
Connector	A mechanism through which components interact
Interface	A defined set of operations or services exposed by a component
Layered architecture	A structure that separates responsibilities into layers
MVC	Model–View–Controller separation of data/domain, presentation, and request handling
Repository	A shared data store accessed by multiple components
Pipe-and-filter	A processing style where data passes through sequential transformations

Term	Meaning
Quality attribute	A non-functional property such as security, performance, or maintainability
Trade-off	A decision that improves some goals while potentially making others harder

12. Homework and Self-Study

1. Draw a layered architecture for a Library Management System. Label each layer and give two responsibilities per layer.

2. Compare MVC and layered architecture. Explain why they can be used together rather than being mutually exclusive.

3. Choose three quality attributes for CEMS and write a measurable scenario for each. State any assumptions.

4. Create a one-page architecture decision record explaining why your team selected its architecture and which alternative it rejected.

5. Extension: describe one situation in which a monolithic application is appropriate and one in which independently deployed services may be justified.

> **Next chapter:** Object-Oriented Design and UML — identifying classes, relationships, and system behavior using diagrams.